Authors: Haim Bar and Assaf Bester
haim.bar@uconn.edu bestera@technion.ac.il

Contents

RNA_lexis – User Guide **1**
Overview . . . 1
Launching the Application . . . 1
Choosing Input . . . 2
Session Management . . . 2
Main Menu . . . 3
Tips . . . 10
Package Structure . . . 10

RNA_lexis – User Guide

Authors: Haim Bar (haim.bar@uconn.edu) and Assaf Bester (bestera@technion.ac.il)

Overview

RNA_lexis is an interactive command-line tool for exploring RNA (or DNA) sequence data. Starting from a nucleotide sequence, it automatically identifies repeating subsequences called **xmotifs** and extracts their shorter conserved cores. From there you can visualize the neighbourhood of any core, study k-mer statistics, generate sequence logos, inspect coverage, and compare subsequences – all from a hierarchical menu driven by numbered choices.

Launching the Application

```
rna_lexis           # after pip install (console script)
python -m rna_lexis # without installing
```

At startup the tool checks the **current working directory** for saved session files (`.json`):

- If **exactly one** valid session file is found, it is loaded automatically — no prompt is shown.
- If **more than one** valid session file is found, a numbered menu lets you choose which one to load.
- If **no** valid session file is found in the current directory, you will be prompted to choose a **data directory** via a folder-browser dialog. The same auto-detection is then applied to the chosen directory.

The data directory is the folder where session files and output files are saved. You can change it later under *Set default values* (see below).

It is recommended that each sequence analysed be stored in its own directory, since output files are stored in the same directory as the sequence, as well as the saved-session JSON file.

Choosing Input

Every session starts by loading a sequence. Four input methods are available:

```
--- Choose Input ---
1. Load from local file
2. Fetch by Ensembl transcript ID (ENST)
3. Paste sequence
4. Python prompt
```

1 - Load from local file

A file-selector dialog opens. Select any plain-text file containing a nucleotide sequence (letters only; spaces and punctuation are stripped automatically). If you select a **`.json`** session file that was saved by a previous run, that session is restored instantly – no re-processing required.

2 - Fetch by Ensembl transcript ID

Enter an Ensembl transcript identifier (e.g. `ENST00000456328` or `ENST00000456328.2`). The tool:

1. Checks whether a cached session for that transcript already exists in your data directory; if so, loads it immediately.
2. Otherwise, queries the Ensembl REST API for the cDNA sequence, displays the transcript name and length, and asks you to choose a folder in which to save the data.

The fetched sequence is stored in a JSON file in the chosen directory and the file name contains the transcript number and name (e.g., ENST00000419829_XIST-204.json).

> **Note:** an active internet connection is required for live fetching.

3 - Paste sequence

Paste one or more lines of sequence text directly into the terminal and press **Enter on an empty line** to finish. You will be prompted for a name for the sequence and a folder in which to save results.

4 - Python prompt

Returns control to the Python interpreter.

Session Management

After loading a sequence the tool automatically:

- **Parses** the text to find xmotifs and cores.
- **Saves a session file** (`<name>.json`) containing the sequence, xmotifs, cores, and summary statistics. On the next run, if this is the only session file in the directory, it is loaded automatically and the parsing step is skipped.
- **Creates a summary CSV** (`<name>_init.csv`) with per-sequence statistics (length, occurrence count, mutation rate, etc.) and opens it in your default spreadsheet application.

To re-open that CSV at any time, use **Open Core file** from the main menu. See below, in the Main Menu section.

Main Menu

```
--- Main Menu ---
1. Plots
2. Sequence operations
3. Summary statistics
4. Set default values
5. Show settings
6. Open Core file
7. Open User Guide
8. Load new input
9. Quit
```

Navigate by typing the option number and pressing **Enter**. Press **Ctrl+D** at any prompt to cancel the current operation and return to the main menu. Press **Ctrl+C** to exit the program immediately.

---

1 - Plots

Opens the Plots submenu:

```
--- Plots ---
1. Core neighbors
2. K-mers
3. Logo
4. Coverage
5. Motif Match/Mutation
6. Back
```

1.1 Core neighbors Visualises how a chosen core sequence co-occurs with its neighbours across the full text.

Prompts:

Prompt	Default	Notes
Sequence to analyse	no default	Must be present in the text
Neighbourhood width	40	Number of positions on each side of the sequence
Strings to include	1 (cores)	1 = cores, 2 = xmotifs
Plot title	`<file> <sequence>`	Free text
Output file name	*(screen only)*	Leave blank to display interactively
Output format	1 (PNG standard)	1 = PNG, 2 = PNG high-res (3×), 3 = SVG, 4 = HTML
X-axis range	*(full range)*	`min, max` e.g. `100, 500`

The interactive HTML plot is always shown on screen. If an output file name is given, the plot is also saved to disk in the chosen format.

Reading the plot:

- The reference sequence (s0) sits at **y = 0**.
- **Neighbouring cores** are shown above (y = 1, 2, …), sorted by how much they overlap with s0. Purple squares = cores that share no positions with s0; green squares/triangles = partial or full overlap.
- **Mutations** of s0 (if any) appear below the axis (y = -1, -2, …), with the closest mutation to s0 (highest occurrence count) nearest to the axis.
- Triangle direction indicates which end of the neighbour extends beyond s0: > extends to the right, < extends to the left.
- A legend at the bottom of the plot explains all symbols and colours.

1.2 K-mers Analyses the frequency distribution of all substrings of a given length k.

Prompts:

Prompt	Default
k-mer length	5

Then choose a plot type:

```
1. Z-score
2. Robust Z-score
3. Abundance histogram
4. Rank-frequency
5. Return to plots
```

The Z-score plot highlights over/under-represented k-mers. The robust version uses a median-based statistics. For Z-score plots you will also be asked for a **threshold** (default 1.96).

The abundance histogram shows the total number of k-mer occurrences in the text.

The Rank-frequency plot shows the log of the rank of each score vs. the log of the number of occurrences. This plot is often used to check whether the distribution of k-mers follows Zipf's law.

After the plot-specific parameters, you will be prompted for an **output file name** and **format** (PNG standard, PNG high-res 3×, or SVG). Leave the file name blank to display on screen only.

1.3 Logo Generates a sequence logo centred on all occurrences of a chosen sequence.

Prompts:

Prompt	Default	Notes
Sequence to analyse	no default	Must be present in the text
Bases on each side	0	Flanking context to include
Max mutations	0	Fuzzy matching; capped at len/6
Format	pdf	`pdf` or `svg`
Output file name	`<seq>_logo`	

1.4 Coverage Shows how much of the text is covered by cores or xmotifs, weighted by a power law.

Prompts:

Prompt	Default	Notes
Strings to include	1 (cores)	1 = cores, 2 = xmotifs
Exponent a	1.2	Score = length^a times occurrences
Output file name	*(screen only)*	Leave blank to display interactively
Output format	1 (PNG standard)	1 = PNG, 2 = PNG high-res (3×), 3 = SVG

1.5 Motif Match/Mutation Plots the positions of up to three user-supplied sequences along the full text as colored rectangles on a shared horizontal lane. Because all sequences share the same lane, overlapping hits from different sequences are visible through the semi-transparent fills.

Prompts:

Prompt	Default	Notes
Sequence 1 / 2 / 3		Leave blank to stop after fewer than 3
Mutation rate: 1 per N letters	7	Sets the maximum number of mismatches per sequence: `floor(L / N)`, capped at 4
Output file	*(screen only)*	Leave blank to display interactively
Output format	1 (PNG)	1 = PNG, 2 = PNG high-res (3×), 3 = SVG, 4 = HTML
Condense x-axis?	N	If `y`, large empty regions on the x-axis are compressed
Minimum gap size to compress	1000	Only gaps wider than this many positions are compressed (shown only when condensing)

Reading the plot:

- Each sequence is drawn in a distinct color: **blue** (seq 1), **orange** (seq 2), **green** (seq 3).
- **Exact matches** are solid rectangles.
- **Approximate matches** (within the mutation allowance) carry thin **red vertical lines** at each mismatched character position inside the rectangle.
- Overlapping rectangles from different sequences are visible because all fills use 55 % opacity.
- The legend shows each sequence with its exact and approximate match counts, plus entries explaining the red line and (if condensed) the dashed separator.
- When condensing is active, gaps exceeding the threshold are replaced by a short stub and a **gray dashed vertical line** marks each compression point. The x-axis ticks show original sequence positions at cluster boundaries.

Optional detailed nucleotide-level view:

After the main plot is shown, you will be asked whether to create a detailed HTML view. If you answer `y`, two more prompts appear:

Prompt	Default	Notes
Position range	full sequence	`start,end` e.g. `200,800`; ranges > 500 nt render slowly
Output HTML file	`<name>_detail.html`	Always saved as HTML; `.html` is appended if omitted

The detail view is a plain HTML page showing every nucleotide in the selected range as monospace text (60 characters per line, prefixed with the genomic position). Hit regions have colored backgrounds matching the main plot. Where two or more sequences overlap, the background is a blended mix of their colors so both are visible simultaneously. Mutation positions are shown in **red uppercase**. A legend lists each sequence with its exact and approximate match counts.

---

2 - Sequence operations

```
--- Sequence operations ---
1. Find all matches
2. Covered area
3. Print core
4. Alignment score for two sequences
5. Search with mutations
6. Extend match pair
7. Export hairpins to CSV
8. Motif extensions
9. Back
```

2.1 Find all matches Searches the full text for a query sequence (minimum 4 characters). Use `.` as a single-character wildcard. Reports the number of matches and prints each match with its position in the text.

2.2 Covered area Computes and prints the coverage score for a single sequence: `length^a times occurrences`, where a is a configurable exponent (default 1).

2.3 Print core Given a sequence, finds and prints the xmotifs that contain it as a core.

2.4 Alignment score for two sequences Aligns two substrings extracted from the loaded text by position.

Prompts:

Prompt	Notes
Start position of first sequence	0-based index into the text
Start position of second sequence	
Sequence length	Both substrings share the same length
Alignment type	1 = Local (Smith–Waterman), 2 = Global (Gotoh)

The alignment is printed with gap symbols and the following scores:

- **Raw score** — the integer alignment score.
- **Normalised score** — raw score divided by the self-alignment score of the shorter sequence; lies in [0, 1]. Values above 0.70 indicate a clearly related pair; above 0.90 near-identical.

- **Bit score** — length-independent score computed with the Karlin–Altschul formula (= 1.28, K = 0.46); comparable across alignments of different lengths.
- **E-value** — expected number of alignments with a score this high or better by chance, using the full transcript as the reference database. Values below 10^{3} are considered significant.

2.5 Search with mutations Searches the full text for a single query string and reports both exact and approximate matches (within a configurable mutation allowance). Mismatched characters in approximate matches are printed in **RED** and **UPPERCASE** so differences are immediately visible.

Prompts:

Prompt	Default	Notes
Query string		Blank to cancel and go back to the menu
Mutation rate: 1 per N letters	7	Maximum number of nucleotide for one mismatch

Output:

- **Exact matches** — count and list of start positions.
- **Approximate matches** — each match printed as `pos <position> "<matched string>" dist=<hamming distance>`, with mutated characters highlighted in red uppercase, e.g. `ugaaacGuac`.

Press **Enter** to return to the Sequence operations menu.

2.6 Extend match pair Finds all exact occurrences of a seed sequence and, for every pair of occurrences, greedily extends both copies left and right as far as possible while keeping the Hamming distance between the two extended copies within a user-defined rate. The result reveals how much context around two repeats remains mutually similar.

How the extension works:

1. The right flank is extended first, one nucleotide at a time. A position is kept whenever the cumulative Hamming distance does not exceed `floor(total_length × mutr)`. The loop stops as soon as the budget for the full remaining text is exhausted.
2. The left flank is then extended using the budget that remains after the right extension.

Prompts:

Prompt	Default	Notes
Seed sequence		Must occur at least twice in the text; blank to cancel
Mutation rate: 1 per N letters	6	1 mismatch allowed per N nucleotides

Output:

Up to 10 pairs are shown, sorted by total extension length (longest first). For each pair:

- Positions of the two seed copies, total extended length, and left/right extension sizes.
- Both extended sequences printed side-by-side: the seed region is shown in **BOLD UPPERCASE**; mismatched characters in the flanks are shown in **RED UPPERCASE**.
- Hamming distance as a count and percentage.

Printing a pair in full:

After the summary, you are offered the option to print one pair without truncation:

```
Print a pair in full [1-10, blank to skip]:
```

Entering a number prints the raw plain-text sequences for each copy (safe to copy), followed by a colour-coded alignment for visual reference.

Saving results to CSV:

```
Save all pairs to CSV [Enter for extensions_<seed>.csv, Ctrl+D to skip]:
```

Press **Enter** to save to the default filename, type a custom name, or press **Ctrl+D** to skip. The CSV contains one row per pair with columns: `rank`, `pos1`, `pos2`, `left_ext`, `right_ext`, `total_len`, `hamming`, `ext1`, `ext2`.

2.8 Motif extensions Combines motif search with pairwise extension analysis. Enter a motif, find all its occurrences (exact or approximate), then automatically compute the longest Hamming-bounded extension for every pair of occurrences and display the results with colour-coded alignments.

Prompts:

Prompt	Default	Notes
Motif		Blank to cancel
Search method	1 (Exact)	`1` = exact (`find_all_matches`); `2` = with mutations (`find_with_mutations`)
Search mutation rate: 1 per N letters	7	Only shown when search method = 2
Extension mutation rate: 1 per N letters	6	Controls how divergent the flanking context may be during extension

Match display:

After the search, all found positions are listed (up to 20):

- **Exact** matches are shown in green with their start position.
- **Approximate** matches (search method 2 only) are shown in red with the Hamming distance.

At least **2 occurrences** (exact or approximate) are required to proceed to the extension step.

Extension output:

Up to 10 pairs are shown, sorted by total extension length (longest first). For each pair:

- Start positions of both seed copies, total extended length, and left/right extension sizes.
- Both extended sequences printed side-by-side (truncated to 80 characters for long extensions, with the full length noted): the seed region is shown in **BOLD UPPERCASE**; mismatched characters in the flanks are shown in **RED UPPERCASE**.
- Hamming distance as a count and percentage.

Printing a pair in full:

After the summary is displayed, you are offered the option to print one pair in full so that the sequences can be selected and copied:

```
Print a pair in full [1-10, blank to skip]:
```

Entering a number prints: - The raw (plain-text) extended sequence for each copy, labelled with its start position — no colour codes, safe to copy directly. - A colour-coded alignment of the same pair (bold seed, red mismatches) for visual reference.

Saving results to CSV:

After the print-in-full step you are prompted to save all pairs to a CSV file:

```
Save all pairs to CSV [Enter for extensions_<motif>.csv, Ctrl+D to skip]:
```

Press **Enter** to save to the default filename (based on the motif), type a custom name, or press **Ctrl+D** to skip. The CSV contains one row per pair with columns: `rank`, `pos1`, `pos2`, `left_ext`, `right_ext`, `total_len`, `hamming`, `ext1`, `ext2`.

---

3 - Summary statistics

Displays counts and lengths derived from the current session:

Statistic	Description
Text length	Total number of characters in the sequence
Number of xmotifs	Count of recurring subsequences found
Longest / Shortest xmotif	Character lengths
Number of cores	Count of conserved core sequences
Longest / Shortest core	Character lengths

Press **Enter** to return to the main menu.

---

4 - Set default values

Lets you change the parameters used when parsing the sequence. Pressing Enter at any prompt keeps the current value.

Parameter	Default	Description
Min xmotif length	7	Shortest subsequence considered as an xmotif
Max xmotif length	40	Longest subsequence considered as an xmotif
Min core length	6	Shortest conserved core
Min occurrences	2	Minimum number of times a pattern must appear
Coverage weight (*pwr*)	1.2	Exponent for the length times occurrence score
Clear screen	True	Whether to clear the terminal between menus
Data directory	*(chosen at startup)*	Default folder for file dialogs

Note: changing min/max xmotif length, min core length, or min occurrences triggers an automatic re-analysis of the loaded text. The session JSON and summary CSV are updated automatically.

Auto-expansion of max xmotif length: if the longest xmotif found during analysis equals the current maximum length, the maximum is automatically raised by 20 and the analysis is re-run. This repeats until the longest xmotif is strictly shorter than the maximum. The expanded value is kept as the new maximum for the session and is shown in *Show settings*.

---

5 - Show settings

Prints the current default values (read-only). Press **Enter** to return.

---

6 - Open Core file

Opens the summary CSV (`<name>_init.csv`) in your default spreadsheet application. If the file is already open, the existing window is brought to the foreground rather than opening a second copy.

---

7 - Open User Guide

Opens this user guide in your default web browser.

---

8 - Load new input

Discards the current session and returns to the **Choose Input** screen so you can work with a different sequence.

---

9 - Quit

Exits the program.

Tips

- **Resuming a session:** launch the tool from the directory that contains the session file (`.json`). If exactly one valid session file is present, it is detected and loaded automatically with no prompts. If multiple session files are present, a selection menu is shown.
- **Cancelling mid-prompt:** press **Ctrl+D** to abandon the current operation and jump back to the main menu without losing your session.
- **Output files:** plots and logos are saved to the working directory (the folder of the loaded file) unless you specify an absolute path.
- **RNA vs DNA:** if the sequence contains only `a`, `c`, `g`, `u` (RNA), uracil (`u`) is automatically converted to thymine (`t`) for internal processing.

Package Structure

The `rna_lexis` package is organised into focused sub-modules. New code should import directly from the relevant module rather than from the top-level `rna_lexis` namespace.

Module	Contents
`rna_lexis.algorithms`	Pure-computation functions — k-mer counting, motif finding, core extraction, hairpin generation, fuzzy matching, alignment scoring helpers. No I/O or plotting dependencies.
`rna_lexis.alignment`	Gotoh global and local alignment with affine gap penalties (`gotoh_global`, `gotoh_local`, `print_alignment`, `AlignmentResult`).
`rna_lexis.plots`	All matplotlib and Plotly visualisation functions — sequence logos, z-score plots, k-mer histograms, frequency-rank plots, coverage plots, sequence-hit diagrams.
`rna_lexis.io`	File and session I/O (`read_text`, `save_session`, `load_session`, `is_valid_session`, `init_summary`), Ensembl REST API fetching (`fetch_enst_cdna`), and platform-aware file opening utilities.
`rna_lexis.dialogs`	Thin tkinter wrappers for native file/directory chooser dialogs (`openFile`, `openDir`).
`rna_lexis.menu`	Interactive CLI menu — all user-facing prompts, menus, and session management.
`rna_lexis.RNAlang`	Backward-compatibility shim. Re-exports every public symbol from the modules above so that existing code using `from rna_lexis.RNAlang import …` or `import rna_lexis.RNAlang as RNA` continues to work without modification.

Example — using sub-modules directly

```
from rna_lexis.algorithms import cores, find_with_mutations
from rna_lexis.alignment import gotoh_global, print_alignment
from rna_lexis.io import read_text, save_session

txt = read_text("my_sequence.txt")
corelist = cores(txt)
result = gotoh_global(corelist[0], corelist[1])
print_alignment(result)
```